



POLITEKNIK NEGERI SEMARANG
JURUSAN TEKNIK ELEKTRO
PROGRAM STUDI STR TEKNOLOGI REKAYASA KOMPUTER

JOBSHEET PRAKTIKUM

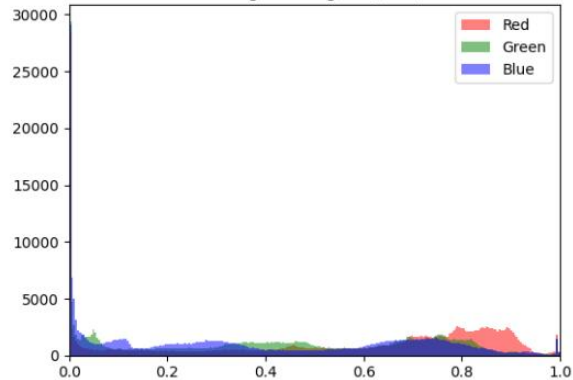
PENGOLAHAN CITRA

JOBSHEET 03: PERBAIKAN CITRA DALAM DOMAIN SPASIAL

Color Original



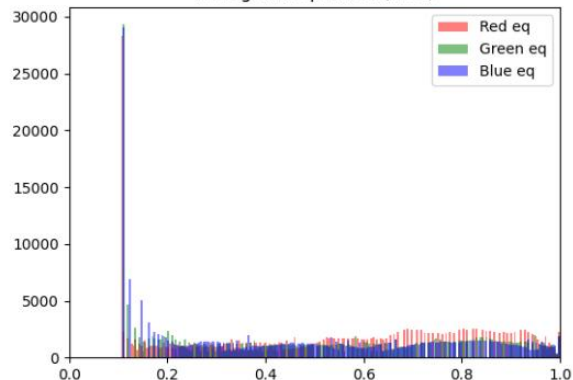
Histogram Original (RGB)



Color Equalized (per channel)



Histogram Equalized (RGB)



Dosen Pengampu:

Ir. Prayitno, S.ST., M.T., Ph.D.

Nama Mahasiswa: _____

NIM: _____

Kelompok: _____



Semester Genap 2025

A. Tujuan Instruksional Khusus

Mahasiswa dapat menguraikan konsep perbaikan citra dalam domain spasial, dengan indikator:

1. Menguraikan konsep transformasi kurva
2. Menjelaskan konsep dasar perbaikan citra pada domain spasial, termasuk jenis-jenis metode yang digunakan.
3. Mengidentifikasi perbedaan antara metode smoothing, sharpening, dan histogram equalization dalam perbaikan citra.
4. Mengimplementasikan metode perbaikan citra pada domain spasial menggunakan perangkat lunak pengolahan citra digital (ScikitImage).
5. Menganalisis hasil perbaikan citra dengan membandingkan kualitas citra sebelum dan sesudah perbaikan secara visual dan menggunakan metrik evaluasi (MSE, PSNR).
6. Menunjukkan ketepatan dalam pemilihan dan penggunaan metode spasial sesuai dengan karakteristik permasalahan citra.

B. Dasar Teori

Perbaikan citra (*image enhancement*) adalah proses dalam pengolahan citra digital yang bertujuan meningkatkan kualitas visual suatu citra sehingga informasi di dalamnya lebih mudah diinterpretasikan. Teknik perbaikan citra dalam **domain spasial** dilakukan dengan memanipulasi langsung nilai intensitas piksel pada citra tersebut.

1. Domain Spasial

Dalam domain spasial, perbaikan citra dilakukan melalui operasi langsung pada piksel citra. Teknik ini melibatkan transformasi nilai intensitas piksel berdasarkan fungsi tertentu atau melalui penerapan filter spasial. Menurut penelitian oleh Singh et al. (2023), metode domain spasial dapat dikategorikan menjadi dua pendekatan utama: berbasis piksel dan berbasis kernel. Pendekatan berbasis piksel memodifikasi nilai intensitas piksel secara individual, sementara pendekatan berbasis kernel menggunakan matriks koefisien untuk memproses sekelompok piksel secara simultan.

2. Transformasi Kontras

Transformasi kontras bertujuan untuk meningkatkan perbedaan antara area terang dan gelap dalam citra, sehingga detail menjadi lebih jelas. Beberapa metode yang umum digunakan meliputi:

- **Peregangan Kontras Linier:** Metode ini memperluas rentang intensitas piksel dalam citra untuk memanfaatkan seluruh rentang dinamis yang tersedia. Hal ini membantu dalam meningkatkan visibilitas detail dalam citra.
- **Histogram Equalization:** Teknik ini mendistribusikan ulang nilai intensitas piksel sehingga histogramnya merata, yang meningkatkan kontras global citra. Menurut penelitian oleh Singh et al. (2023), metode ini efektif dalam meningkatkan kontras citra dengan menyamakan distribusi intensitas piksel.

- **Adaptive Histogram Equalization (AHE):** Berbeda dengan histogram equalization global, AHE bekerja pada area lokal citra, memungkinkan peningkatan kontras pada daerah dengan variasi kontras yang berbeda-beda. Namun, metode ini dapat memperkuat noise jika tidak diterapkan dengan hati-hati.

3. Filtering dalam Domain Spasial

Filtering digunakan untuk mereduksi noise dan meningkatkan detail dalam citra. Beberapa jenis filter yang umum digunakan meliputi:

- **Smoothing Filters:**
 - *Mean Filter:* Menggantikan nilai piksel dengan rata-rata nilai piksel di sekitarnya, efektif untuk mereduksi noise acak.
 - *Gaussian Filter:* Menggunakan distribusi Gaussian untuk memberikan pembobotan pada piksel di sekitarnya, menghasilkan efek smoothing yang lebih alami.
 - *Median Filter:* Menggantikan nilai piksel dengan median dari nilai piksel di sekitarnya, sangat efektif untuk menghilangkan noise impulsif seperti salt-and-pepper.
- **Sharpening Filters:**
 - *Laplacian Filter:* Menyoroti area dengan perubahan intensitas yang cepat, seperti tepi objek, sehingga meningkatkan ketajaman citra.
 - *Unsharp Masking:* Metode ini meningkatkan ketajaman dengan menambahkan kembali detail tinggi yang telah dikurangi melalui proses smoothing sebelumnya.

C. Alat dan Bahan

Untuk dapat melaksanakan praktik jobsheet ini diperlukan:

- Komputer dengan Python terinstal
- Pustaka Python: scikit-image, matplotlib, numpy
- Contoh gambar dari scikit-image

D. Langkah Kerja Praktik

Praktikum 1. Transformasi Kurva (Citra Negatif)

Transformasi Kurva (Negatif): Citra negatif yang dihasilkan menampilkan tonal yang terbalik dari citra asli. Area yang semula terang menjadi gelap, dan area gelap menjadi terang. Hal ini sesuai dengan konsep transformasi negatif yang melakukan pembalikan nilai intensitas (misalnya, piksel bernilai 200 pada citra 8-bit akan menjadi 55 pada citra negatif, karena $255 - 200 = 55$). Transformasi kurva seperti ini tidak mengubah detail spasial, namun mengubah tampilan visual citra. Dalam praktik, citra negatif dapat memudahkan pengamatan detail tertentu yang mungkin kurang terlihat pada citra aslinya (misalnya pada foto rontgen).

Kode Praktikum:

```
01: import numpy as np
02: import matplotlib.pyplot as plt
03: from skimage import data, img_as_float
04:
05: # =====
06: # 1. TRANSFORMASI NEGATIF CITRA GRAYSCALE
07: # =====
08: image_gray_uint8 = data.camera() # Citra grayscale (512x512, 0..255)
09: image_gray_float = img_as_float(image_gray_uint8) # Konversi ke float [0..1]
10:
11: negative_gray = 1.0 - image_gray_float # Inversi intensitas: output = 1 - input
12:
13: # =====
14: # 2. TRANSFORMASI NEGATIF CITRA BERWARNA
15: # =====
16: image_color_uint8 = data.astronaut() # Citra berwarna (512x512x3, 0..255)
17: image_color_float = img_as_float(image_color_uint8) # Konversi ke float [0..1]
18:
19: negative_color = 1.0 - image_color_float # Inversi intensitas tiap kanal RGB
20:
21: # =====
22: # 3. PLOTTING HASIL
23: # =====
24: fig, axes = plt.subplots(2, 2, figsize=(12, 8))
25:
26: # Baris 1: Grayscale (original, negative)
27: axes[0, 0].imshow(image_gray_float, cmap='gray')
28: axes[0, 0].set_title("Grayscale (Original)")
29: axes[0, 0].axis('off')
30:
31: axes[0, 1].imshow(negative_gray, cmap='gray')
32: axes[0, 1].set_title("Grayscale (Negative)")
33: axes[0, 1].axis('off')
34:
35: # Baris 2: RGB (original, negative)
36: axes[1, 0].imshow(image_color_float)
37: axes[1, 0].set_title("Color (Original)")
38: axes[1, 0].axis('off')
39:
40: axes[1, 1].imshow(negative_color)
41: axes[1, 1].set_title("Color (Negative)")
42: axes[1, 1].axis('off')
43:
44: plt.tight_layout()
45: plt.show()
46:
```

Hasil Praktikum P1.

Grayscale (Original)



Grayscale (Negative)



Color (Original)



Color (Negative)



Praktikum 2. Histogram dan Ekualisasi Histogram

Histogram dan Ekualisasi Histogram: Setelah dilakukan ekualisasi histogram, kontras citra meningkat terutama pada bagian-bagian yang tadinya gelap. Pada contoh citra moon, detail kawah yang semula kurang terlihat menjadi lebih jelas setelah ekualisasi. Histogram citra hasil ekualisasi tersebar lebih merata di seluruh rentang intensitas, tidak lagi terkumpul di ujung bawah saja. Ini menandakan rentang intensitas yang tadinya sempit telah diperlebar. Dengan kata lain, ekualisasi histogram berhasil meratakan distribusi intensitas sehingga area gelap menjadi lebih terang dan area terang sedikit lebih gelap, menghasilkan citra dengan detail yang lebih kaya. Perlu dicatat bahwa ekualisasi histogram bersifat global; pada beberapa kasus, metode ini bisa menyebabkan hilangnya detail halus pada area yang sudah cukup kontras (over-enhancement), namun pada contoh ini perbaikan kontras bersifat positif.

1. Histogram Citra Grayscale Histogram menampilkan sebaran intensitas (0..1) pada sumbu-x dan jumlah/frekuensi piksel pada sumbu-y. Pada citra grayscale, mudah dilihat jika piksel lebih banyak terkonsentrasi pada nilai gelap atau terang.
2. Ekualisasi Histogram (Grayscale) Fungsi `exposure.equalize_hist(image_gray)` mencoba meratakan histogram citra sehingga area gelap atau terang lebih terbuka kontrasnya. Hasil ekualisasi sering terlihat jelas pada citra yang awalnya terlalu gelap/terang; detail menjadi lebih tampak.

3. Histogram Citra Berwarna Citra RGB terdiri atas 3 kanal: Red (R), Green (G), dan Blue (B). Masing-masing kanal memiliki histogram sendiri. Dengan menumpuk (overlay) ketiga histogram (warna berbeda), kita bisa melihat dominasi intensitas merah, hijau, atau biru.
4. Ekualisasi Histogram (Citra Berwarna) Contoh di atas melakukan ekualisasi histogram pada setiap kanal secara independen. Hasilnya, kontras dalam masing-masing kanal diperbaiki, tetapi keseimbangan warna dapat berubah. Pendekatan lain adalah mengubah citra ke color space Lab* atau HSV, lalu mengekualisasi hanya kanal luminance (L) atau value (V). Hal ini sering memberikan hasil yang lebih alami secara visual dibanding ekualisasi per kanal.

Kode Praktikum:

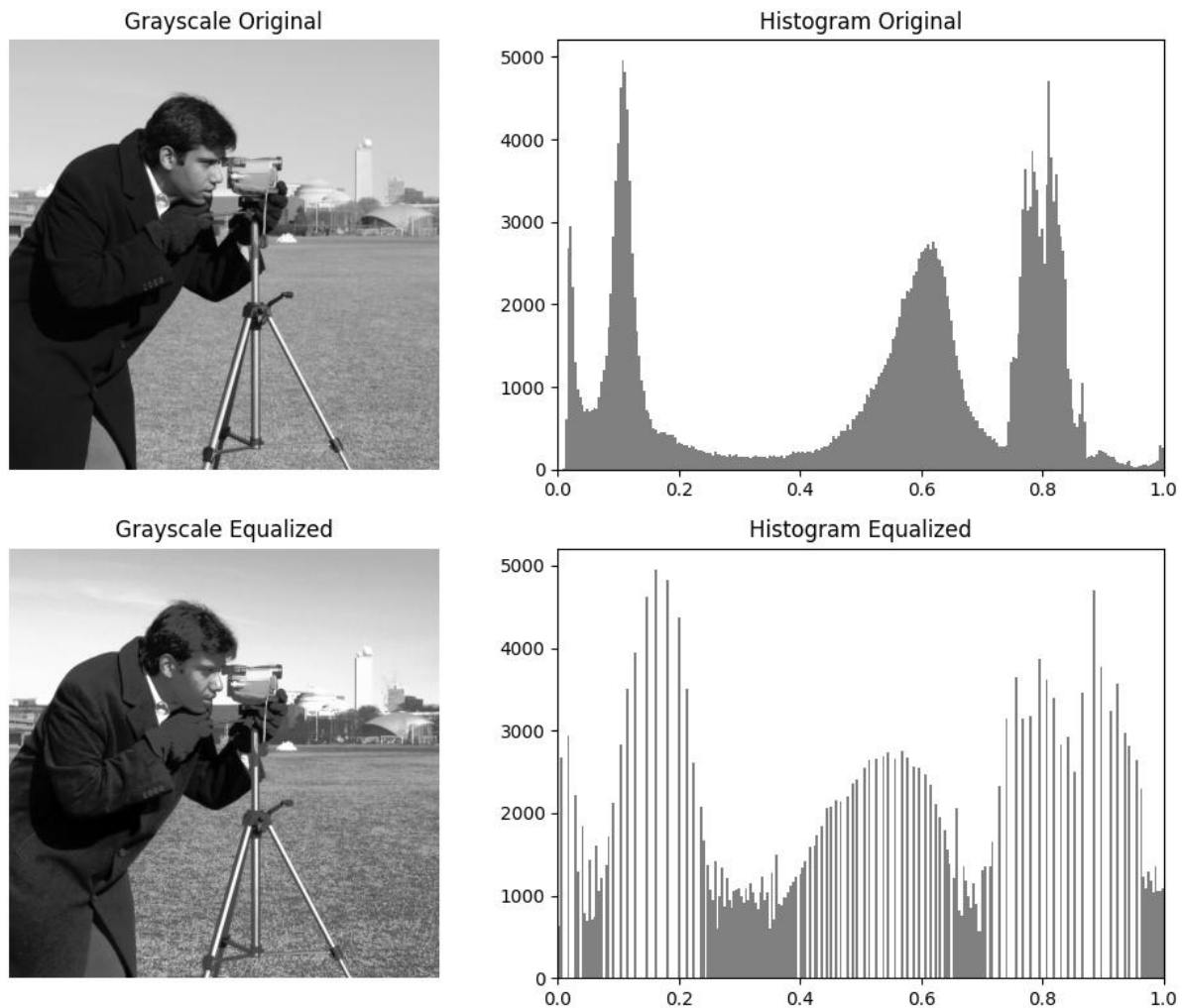
```
001: import numpy as np
002: import matplotlib.pyplot as plt
003: from skimage import data, exposure, img_as_float
004:
005: # -----
006: # BAGIAN 1: HISTOGRAM & EKUALISASI HISTOGRAM (GRAYSCALE)
007: # -----
008:
009: # 1. Memuat citra grayscale
010: image_gray_uint8 = data.camera() # sudah grayscale (uint8, 0..255)
011: image_gray = img_as_float(image_gray_uint8) # konversi ke float [0..1]
012:
013: # 2. Menghitung histogram citra asli (dengan numpy.histogram)
014: # Kita perbesar bins=256 untuk melihat distribusi piksel
015: hist_gray, bins_gray = np.histogram(image_gray.ravel(), bins=256,
range=(0,1))
016:
017: # 3. Ekualisasi histogram
018: # Dengan skimage.exposure.equalize_hist, hasil di rentang [0..1]
019: image_gray_eq = exposure.equalize_hist(image_gray)
020: hist_gray_eq, bins_gray_eq = np.histogram(image_gray_eq.ravel(),
bins=256, range=(0,1))
021:
022: # 4. Plot grayscale: citra asli & histogram, citra ekualisasi & histogram
023: fig, axes = plt.subplots(2, 2, figsize=(10, 8))
024:
025: # (a) Citra Asli Grayscale
026: axes[0, 0].imshow(image_gray, cmap='gray')
027: axes[0, 0].set_title("Grayscale Original")
028: axes[0, 0].axis('off')
029:
030: # (b) Histogram Citra Asli
031: axes[0, 1].hist(image_gray.ravel(), bins=256, range=(0,1),
color='gray')
032: axes[0, 1].set_title("Histogram Original")
033: axes[0, 1].set_xlim(0,1)
034:
035: # (c) Citra Hasil Ekualisasi
036: axes[1, 0].imshow(image_gray_eq, cmap='gray')
```

```
037: axes[1, 0].set_title("Grayscale Equalized")
038: axes[1, 0].axis('off')
039:
040: # (d) Histogram Citra Hasil Ekualisasi
041: axes[1, 1].hist(image_gray_eq.ravel(), bins=256, range=(0,1),
color='gray')
042: axes[1, 1].set_title("Histogram Equalized")
043: axes[1, 1].set_xlim(0,1)
044:
045: plt.tight_layout()
046: plt.show()
047:
048: # -----
049: # BAGIAN 2: HISTOGRAM & EKUALISASI HISTOGRAM (CITRA WARNA)
050: # -----
051:
052: # 1. Memuat citra berwarna
053: image_color_uint8 = data.astronaut() # citra berwarna RGB (uint8)
054: image_color = img_as_float(image_color_uint8) # konversi ke float
[0..1]
055:
056: # 2. Kita dapat menghitung histogram gabungan (semua kanal) atau per
kanal
057: # Di sini kita tampilkan histogram gabungan R, G, B untuk citra asli.
058: r_channel = image_color[:, :, 0]
059: g_channel = image_color[:, :, 1]
060: b_channel = image_color[:, :, 2]
061:
062: # Hitung histogram untuk tiap kanal, 256 bins
063: hist_r, bins_r = np.histogram(r_channel.ravel(), bins=256, range=(0,1))
064: hist_g, bins_g = np.histogram(g_channel.ravel(), bins=256, range=(0,1))
065: hist_b, bins_b = np.histogram(b_channel.ravel(), bins=256, range=(0,1))
066:
067: # 3. Ekualisasi histogram tiap kanal secara independen
068: # (Catatan: cara ini bisa mengubah keseimbangan warna.
069: # Terkadang lebih baik melakukan ekualisasi hanya pada kanal
luminance (Y)
070: # atau Value (HSV). Tapi di sini kita tunjukkan contoh per kanal.)
071: r_eq = exposure.equalize_hist(r_channel)
072: g_eq = exposure.equalize_hist(g_channel)
073: b_eq = exposure.equalize_hist(b_channel)
074:
075: # Gabungkan kembali menjadi citra RGB yang sudah di-ekualisasi
076: image_color_eq = np.dstack((r_eq, g_eq, b_eq))
077:
078: # Hitung histogram citra berwarna ekualisasi (tiap kanal)
079: hist_r_eq, bins_r_eq = np.histogram(r_eq.ravel(), bins=256,
range=(0,1))
080: hist_g_eq, bins_g_eq = np.histogram(g_eq.ravel(), bins=256,
range=(0,1))
081: hist_b_eq, bins_b_eq = np.histogram(b_eq.ravel(), bins=256,
range=(0,1))
082:
```



```
083: # 4. Plot citra berwarna & histogram
084: fig2, axes2 = plt.subplots(2, 2, figsize=(10, 8))
085:
086: # (a) Citra Asli (Color)
087: axes2[0, 0].imshow(image_color)
088: axes2[0, 0].set_title("Color Original")
089: axes2[0, 0].axis('off')
090:
091: # (b) Histogram Asli (tumpuk R, G, B)
092: axes2[0, 1].hist(r_channel.ravel(), bins=256, range=(0,1), color='red',
alpha=0.5, label='Red')
093: axes2[0, 1].hist(g_channel.ravel(), bins=256, range=(0,1),
color='green', alpha=0.5, label='Green')
094: axes2[0, 1].hist(b_channel.ravel(), bins=256, range=(0,1),
color='blue', alpha=0.5, label='Blue')
095: axes2[0, 1].set_title("Histogram Original (RGB)")
096: axes2[0, 1].set_xlim(0,1)
097: axes2[0, 1].legend(loc='upper right')
098:
099: # (c) Citra Hasil Ekualisasi (RGB)
100: axes2[1, 0].imshow(image_color_eq)
101: axes2[1, 0].set_title("Color Equalized (per channel)")
102: axes2[1, 0].axis('off')
103:
104: # (d) Histogram Ekualisasi (tumpuk R, G, B)
105: axes2[1, 1].hist(r_eq.ravel(), bins=256, range=(0,1), color='red',
alpha=0.5, label='Red eq')
106: axes2[1, 1].hist(g_eq.ravel(), bins=256, range=(0,1), color='green',
alpha=0.5, label='Green eq')
107: axes2[1, 1].hist(b_eq.ravel(), bins=256, range=(0,1), color='blue',
alpha=0.5, label='Blue eq')
108: axes2[1, 1].set_title("Histogram Equalized (RGB)")
109: axes2[1, 1].set_xlim(0,1)
110: axes2[1, 1].legend(loc='upper right')
111:
112: plt.tight_layout()
113: plt.show()
114:
```

Hasil Praktikum



Praktikum 3. Konvolusi (Filter Rata-rata)

Konvolusi (Filter Rata-Rata): Penerapan filter rata-rata 5×5 pada citra menghasilkan efek smoothing yang jelas terlihat. Citra output tampak lebih kabur dibanding citra original – tepi objek menjadi kurang tajam dan tekstur halus tersamar. Hal ini terjadi karena konvolusi dengan kernel rata-rata menghitung nilai piksel keluaran sebagai rata-rata dari tetangga sekitarnya, sehingga perbedaan ekstrem dihaluskan. Keuntungan dari smoothing ini adalah derau acak atau detail kecil yang tidak diinginkan berkurang. Namun, trade-off-nya adalah kehilangan ketajaman pada tepi objek. Dengan kata lain, filter linear sederhana seperti rata-rata mengorbankan ketajaman demi mengurangi noise. Dalam konteks perbaikan citra, filter ini berguna untuk prapemrosesan (misal mengurangi noise sebelum analisis lebih lanjut), tetapi kurang cocok bila detail tepi harus dipertahankan.

Kode Praktikum:

```
01: import numpy as np
02: import matplotlib.pyplot as plt
03: from skimage import data, img_as_float
04: from scipy import ndimage as ndi
```

```
05:
06: # -----
07: # 1) FILTER RATA-RATA PADA CITRA GRAYSCALE
08: # -----
09:
10: # A. Memuat citra grayscale (camera)
11: gray_uint8 = data.camera() # range 0..255
12: gray_img = img_as_float(gray_uint8) # konversi menjadi float [0..1]
    untuk kemudahan
13:
14: # B. Definisikan kernel filter rata-rata (contoh: 5x5)
15: kernel_size = 5
16: kernel = np.ones((kernel_size, kernel_size), dtype=float) / (kernel_size
    * kernel_size)
17:
18: # C. Lakukan konvolusi pada citra grayscale
19: gray_filtered = ndi.convolve(gray_img, kernel, mode='reflect')
20:
21: # D. Tampilkan hasil
22: fig, axes = plt.subplots(1, 2, figsize=(10, 5))
23: axes[0].imshow(gray_img, cmap='gray')
24: axes[0].set_title("Grayscale - Original")
25: axes[0].axis('off')
26:
27: axes[1].imshow(gray_filtered, cmap='gray')
28: axes[1].set_title(f"Grayscale - Filtered ({kernel_size}x{kernel_size}
    Average)")
29: axes[1].axis('off')
30:
31: plt.tight_layout()
32: plt.show()
33:
34: # -----
35: # 2) FILTER RATA-RATA PADA CITRA BERWARNA
36: # -----
37:
38: # A. Memuat citra berwarna (astronaut)
39: color_uint8 = data.astronaut() # citra RGB (uint8, 0..255)
40: color_img = img_as_float(color_uint8) # konversi float [0..1]
41:
42: # B. Kita akan terapkan filter rata-rata pada tiap kanal R, G, B
43: #     secara terpisah, lalu gabungkan.
44: color_filtered = np.zeros_like(color_img)
45:
46: for c in range(3): # c = 0 (Red), 1 (Green), 2 (Blue)
47:     channel_original = color_img[:, :, c]
48:     channel_filtered = ndi.convolve(channel_original, kernel,
    mode='reflect')
49:     color_filtered[:, :, c] = channel_filtered
50:
51: # C. Tampilkan hasil
52: fig2, axes2 = plt.subplots(1, 2, figsize=(10, 5))
53: axes2[0].imshow(color_img)
```

```
54: axes2[0].set_title("Color - Original")
55: axes2[0].axis('off')
56:
57: axes2[1].imshow(color_filtered)
58: axes2[1].set_title(f"Color - Filtered ({kernel_size}x{kernel_size}
Average)")
59: axes2[1].axis('off')
60:
61: plt.tight_layout()
62: plt.show()
63:
```

Hasil Praktikum:

Grayscale - Original



Grayscale - Filtered (5x5 Average)



Color - Original



Color - Filtered (5x5 Average)



Praktikum 4. Filter Median (penghilangan noise/derau)

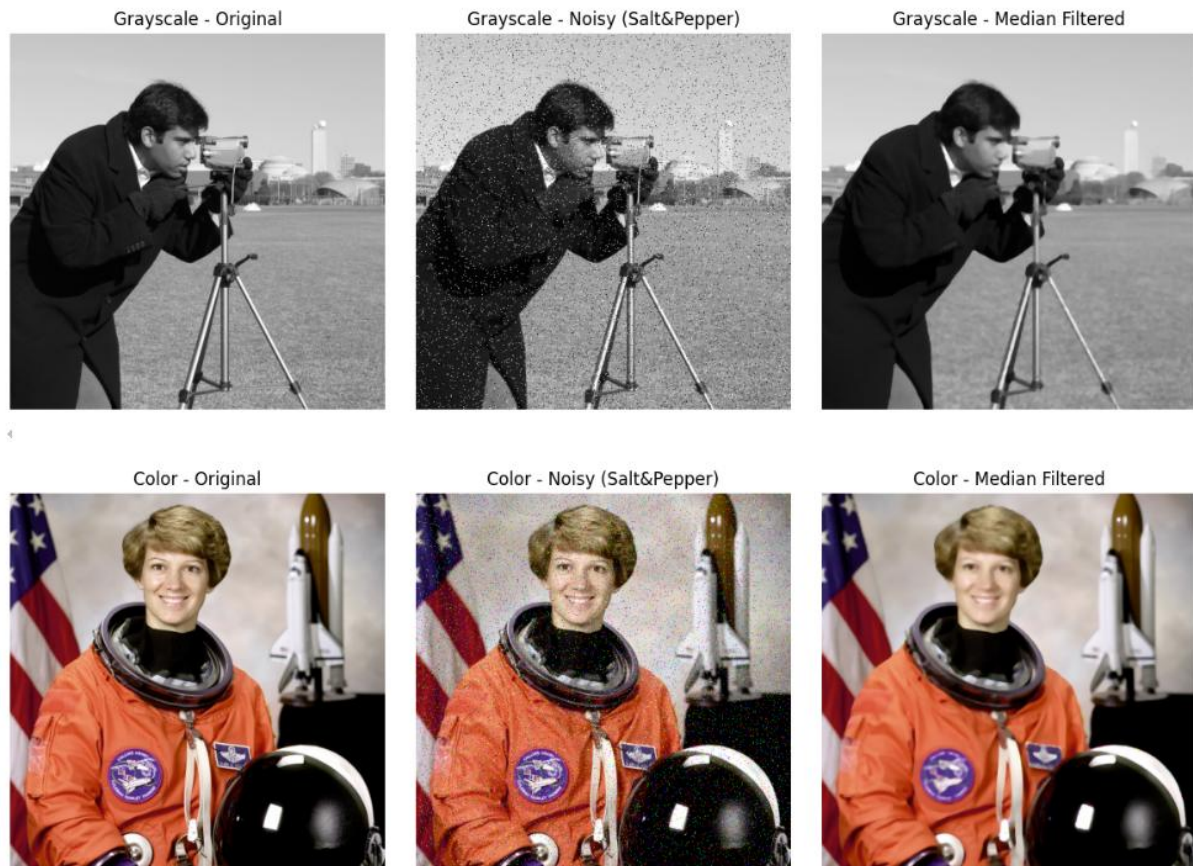
Filter Median (Penghilangan Derau): Hasil penyaringan dengan filter median menunjukkan derau salt-and-pepper berhasil dihilangkan secara efektif. Pada citra berderau, terdapat banyak piksel hitam dan putih acak yang mengganggu (misalnya bintik putih di area gelap dan bintik hitam di area terang). Setelah filtrasi median, bintik-bintik tersebut hampir seluruhnya hilang, sementara detail citra (termasuk tepi objek) relatif terjaga. Berbeda dari filter rata-rata, filter median tidak mengaburkan citra secara signifikan – ini dapat dilihat dari tepi objek (misalnya tepi tepi pada citra camera) yang masih tampak jelas pasca filtering. Hal ini sesuai dengan teori bahwa filter median cenderung menjaga pinggiran saat menghilangkan noise ID.WIKIPEDIA.ORG . Dengan demikian, filter median sangat cocok untuk membersihkan derau impuls pada citra sebelum tahap pengolahan lebih lanjut (misalnya sebelum segmentasi atau deteksi tepi), tanpa terlalu merusak detail penting.

Kode Praktikum:

```
01: import numpy as np
02: import matplotlib.pyplot as plt
03: from skimage import data, util, filters, morphology, img_as_float
04:
05: # -----
06: # 1) FILTER MEDIAN PADA CITRA GRAYSCALE
07: # -----
08:
09: # A. Memuat citra grayscale (contoh: 'camera')
10: gray_uint8 = data.camera() # grayscale, uint8
11: gray_float = img_as_float(gray_uint8) # konversi float [0..1]
12:
13: # B. Menambahkan noise jenis salt-and-pepper
14: gray_noisy = util.random_noise(gray_float, mode='s&p', amount=0.05)
15: # amount=0.05 berarti 5% piksel terpengaruh noise
16:
17: # C. Menerapkan filter median
18: # Kita gunakan structuring element berbentuk disk radius 3
19: disk3 = morphology.disk(3)
20: gray_denoised = filters.median(gray_noisy, disk3)
21:
22: # D. Plot hasil
23: fig, axes = plt.subplots(1, 3, figsize=(12, 4))
24: axes[0].imshow(gray_float, cmap='gray')
25: axes[0].set_title("Grayscale - Original")
26: axes[0].axis('off')
27:
28: axes[1].imshow(gray_noisy, cmap='gray')
29: axes[1].set_title("Grayscale - Noisy (Salt&Pepper)")
30: axes[1].axis('off')
31:
32: axes[2].imshow(gray_denoised, cmap='gray')
33: axes[2].set_title("Grayscale - Median Filtered")
34: axes[2].axis('off')
35:
36: plt.tight_layout()
37: plt.show()
```

```
38:
39: # -----
40: # 2) FILTER MEDIAN PADA CITRA BERWARNA
41: # -----
42:
43: # A. Memuat citra berwarna (contoh: 'astronaut')
44: color_uint8 = data.astronaut()      # RGB, uint8
45: color_float = img_as_float(color_uint8) # rentang [0..1]
46:
47: # B. Menambahkan noise salt-and-pepper
48: color_noisy = util.random_noise(color_float, mode='s&p', amount=0.05)
49:
50: # C. Menerapkan filter median
51: # Pada scikit-image 0.19+, median() mampu menangani array 3D (RGB),
52: # atau kita bisa memanggil per kanal. Di sini langsung:
53: color_denoised = np.zeros_like(color_noisy)
54: for c in range(color_noisy.shape[2]):
55:     color_denoised[:, :, c] = filters.median(color_noisy[:, :, c],
morphology.disk(3))
56:
57: # D. Plot hasil
58: fig2, axes2 = plt.subplots(1, 3, figsize=(12, 4))
59: axes2[0].imshow(color_float)
60: axes2[0].set_title("Color - Original")
61: axes2[0].axis('off')
62:
63: axes2[1].imshow(color_noisy)
64: axes2[1].set_title("Color - Noisy (Salt&Pepper)")
65: axes2[1].axis('off')
66:
67: axes2[2].imshow(color_denoised)
68: axes2[2].set_title("Color - Median Filtered")
69: axes2[2].axis('off')
70:
71: plt.tight_layout()
72: plt.show()
73:
```

Hasil Praktik



E. Hasil Praktikum

Lengkapi tabel hasil praktikum berikut:

No.	Nama Praktikum	Hasil Praktikum
1	Transformasi Kurva (Citra Negatif)	
2	Histogram dan Ekualisasi Histogram	
3	Konvolusi (Filter Rata-rata)	
4	Filter Median (penghilangan noise/derau)	

F. Penugasan

1. Kumpulkan Laporan Praktikum dari jobsheet ini dalam bentuk Microsoft word sesuai dengan format jobsheet praktikum dan dikumpulkan di web elnino polines. (JANGAN DALAM BENTUK PDF)
2. Kumpulkan luaran kode praktikum dalam bentuk ipynb yang sudah diunggah pada akun github masing-masing. Lampirkan tautan github yang sudah di unggah melalui laman LMS elnino.

3. **Eksperimen Transformasi Intensitas Lain:** Cobalah menerapkan transformasi kurva lain selain citra negatif, misalnya transformasi logaritmik atau transformasi gamma. Gunakan citra grayscale yang berbeda (contoh: `skimage.data.camera()` vs `skimage.data.astronaut()` yang diubah ke grayscale). Amati perbedaan yang dihasilkan terhadap citra asli. Jelaskan bagaimana parameter (konstanta c atau gamma) memengaruhi hasil citra.
https://scikit-image.org/docs/stable/api/skimage.exposure.html#skimage.exposure.adjust_gamma
4. **Perbandingan Metode Peningkatan Kontras:** Bandingkan dua metode peningkatan kontras pada citra yang sama – misalnya peregangan kontras (contrast stretching) vs ekualisasi histogram. Gunakan citra grayscale dengan kontras rendah (citra yang tampak terlalu gelap atau terlalu terang). Terapkan kedua teknik tersebut (Anda bisa menggunakan fungsi `exposure.rescale_intensity` untuk contrast stretching, dan `exposure.equalize_hist` untuk histogram equalization). Tampilkan dan bandingkan hasilnya serta histogramnya. Berikan analisis: metode mana yang lebih efektif meningkatkan kontras pada citra tersebut, dan apa perbedaan karakteristik hasil kedua citra.
https://scikit-image.org/docs/stable/api/skimage.exposure.html#skimage.exposure.rescale_intensity
5. **Filter Median vs Filter Rata-rata:** Uji kinerja filter median dibandingkan filter rata-rata dalam mengatasi derau. Tambahkan noise salt-and-pepper pada sebuah citra grayscale (gunakan fungsi `util.random_noise`). Kemudian, saring citra berderau tersebut dengan filter rata-rata (dapat menggunakan konvolusi kernel ones atau `filters.gaussian` untuk smoothing Gaussian) dan dengan filter median. Tampilkan hasil kedua filter dan bandingkan. Fokus pada perbedaan di tepi objek dan sisa noise yang mungkin ada. Jelaskan mengapa salah satu filter lebih unggul dari yang lain untuk kasus noise impuls.
https://scikit-image.org/docs/stable/api/skimage.util.html#skimage.util.random_noise

G. Kesimpulan

Perbaikan citra dalam domain spasial merupakan pendekatan fundamental dalam pengolahan citra digital yang bertujuan untuk meningkatkan kualitas visual melalui manipulasi langsung nilai intensitas piksel. Teknik-teknik seperti peregangan kontras linier, histogram equalization, dan adaptive histogram equalization efektif dalam meningkatkan kontras dan memperjelas detail citra. Selain itu, penggunaan filter smoothing seperti mean, Gaussian, dan median membantu dalam mereduksi noise, sementara filter sharpening seperti Laplacian dan unsharp masking meningkatkan ketajaman detail. Evaluasi kualitas hasil perbaikan citra dapat dilakukan menggunakan metrik seperti Mean Squared Error (MSE) dan Peak Signal-to-Noise Ratio (PSNR). Implementasi teknik-teknik ini dapat dilakukan secara efisien menggunakan pustaka Python seperti Scikit-Image, yang menyediakan berbagai fungsi untuk pemrosesan dan analisis citra digital. Dengan memahami dan menerapkan metode perbaikan citra dalam domain spasial, diharapkan mahasiswa mampu meningkatkan kualitas citra sesuai kebutuhan aplikasi tertentu.

H. Referensi

Munir, Rinaldi. Pengolahan Citra Digital – Bab 7: Perbaikan Kualitas Citra, Institut Teknologi Bandung. (Menjelaskan definisi peningkatan kualitas citra secara matematis dan konsep-konsep dasar)

Faradilla, Putri, et al. (2022). “Implementasi Metode Kernel Konvolusi dan Contrast Stretching untuk Perbaikan Kualitas Citra Digital.” Jurnal Sistem Informasi TGD, Vol. 1 No. 6, 865-874. (Sumber konsep point processing dan konvolusi pada domain spasial)

Wikipedia (Bahasa Indonesia). “Perataan histogram.” (Menjelaskan konsep histogram citra dan teknik ekualisasi histogram sebagai metode peningkatan kontras)

Wikipedia (Bahasa Indonesia). “Filter median.” (Menjelaskan prinsip filter median sebagai penyaring non-linear untuk menghilangkan derau impuls, serta keunggulannya dalam menjaga tepi citra)

.